

Experiment-8

Name: -P. Akhila

Regno: -192472320

Course Code: -MLA0305

Slot-B

Title

Comparative Analysis of SARSA and Q-Learning for Reinforcement Learning

Aim

To implement and compare the **SARSA** and **Q-Learning** algorithms for solving a Reinforcement Learning problem and analyze their performance based on reward, convergence, success rate, and optimal policy.

Procedure

1. Define a Reinforcement Learning environment with a finite set of states and actions.
2. Define the transition function and rewards for each state-action transition.
3. Initialize separate **Q-tables** for SARSA and Q-Learning.
4. Set the learning rate , discount factor , and exploration rate .
5. Use an **ϵ -greedy policy** to balance exploration and exploitation.

6. For **SARSA**:

- Select the current action using the ϵ -greedy policy. ○ Execute the action and observe the next state and reward. ○ Select the next action using the same policy.
- Calculate the TD target:

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

- Update the Q-value using:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

7. For **Q-Learning**:

- Select an action using the ϵ -greedy policy. ○ Execute the action and observe the next state and reward. ○ Select the maximum Q-value of the next state.
- Calculate the TD target:

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

- Update the Q-value using:

$$Q(St, At) \leftarrow Q(St, At) + \alpha[Rt + 1 + \gamma \max_a Q(St + 1, a) - Q(St, At)]$$

8. Repeat the training process for the specified number of episodes.
9. Record rewards, TD errors, steps, and success for both algorithms.
10. Extract the optimal policy using:

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$
11. Compare the learned Q-tables and optimal policies.
12. Compare SARSA and Q-Learning using reward, success rate, TD error, episode length, and execution time.
13. Generate convergence and performance graphs.
14. Display summary tables and export the results to CSV files.
15. Analyze the difference between the **on-policy SARSA** and **off-policy Q-Learning** approaches.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import time
print("=" * 75)
print("EXPERIMENT 8")
print("COMPARATIVE ANALYSIS OF SARSA AND Q-LEARNING")
print("=" * 75)
print("\uAIM")
print("-" * 75)
print(
    "To implement and compare SARSA and Q-Learning algorithms"
)
print(
    "for solving a Reinforcement Learning environment."
)
```

```

print("\nTHEORY")
print("-" * 75)
print("SARSA is an on-policy Temporal Difference algorithm.")
print("Q-Learning is an off-policy Temporal Difference algorithm.")
print("\nSARSA Equation:")
print(
    "Q(S,A) = Q(S,A) + alpha * "
    "[R + gamma*Q(S',A') - Q(S,A)]"
)
print("\nQ-Learning Equation:")
print(
    "Q(S,A) = Q(S,A) + alpha * "
    "[R + gamma*max(Q(S',A')) - Q(S,A)]"
)
states = [
    "Start",
    "S1",
    "S2",
    "S3",
    "Goal"
]
actions = [
    "Left",
    "Right"
]
n_states = len(states)
n_actions = len(actions)
alpha = 0.10
gamma = 0.90
epsilon = 0.20

```

```
episodes = 1000
max_steps = 20
def get_next_state(state, action):
    if state == 0:
        if action == 0:
            return 0, -1
        return 1, 0
    elif state == 1:
        if action == 0:
            return 0, -1
        return 2, 0
    elif state == 2:
        if action == 0:
            return 1, -1
        return 3, 0
    elif state == 3:
        if action == 0:
            return 2, -1
        return 4, 10
    else:
        return 4, 0
def choose_action(q_table, state):
    if random.random() < epsilon:
        return random.randint(
            0,
            n_actions - 1
        )
    return int(
        np.argmax(q_table[state])
    )
```

```
Q_sarsa = np.zeros(
    (n_states, n_actions)
)
Q_qlearning = np.zeros(
    (n_states, n_actions)
)
sarsa_rewards = []
qlearning_rewards = []
sarsa_steps = []
qlearning_steps = []
sarsa_success = []
qlearning_success = []
sarsa_td_errors = []
qlearning_td_errors = []
sarsa_value_history = []
qlearning_value_history = []
sarsa_start = time.time()
for episode in range(episodes):
    state = 0
    action = choose_action(
        Q_sarsa,
        state
    )
    total_reward = 0
    steps = 0
    success = 0
    for step in range(max_steps):
        steps += 1
        next_state, reward = get_next_state(
            state,
```

```
        action
    )
    next_action = choose_action(
        Q_sarsa,
        next_state
    )
    td_target = (
        reward
        + gamma * Q_sarsa[
            next_state,
            next_action
        ]
    )
    td_error = (
        td_target
        - Q_sarsa[state, action]
    )
    Q_sarsa[state, action] += (
        alpha * td_error
    )
    sarsa_td_errors.append(
        td_error
    )
    total_reward += reward
    state = next_state
    action = next_action
    if state == 4:
        success = 1
        break
    sarsa_rewards.append(
```

```

        total_reward
    )
    sarsa_steps.append(
        steps
    )
    sarsa_success.append(
        success
    )
    sarsa_value_history.append(
        Q_sarsa.copy()
    )
sarsa_time = time.time() - sarsa_start
print("\nSARSA Training Completed.")
print(
    "SARSA Execution Time :",
    round(sarsa_time, 5),
    "seconds"
)
qlearning_start = time.time()
for episode in range(episodes):
    state = 0
    total_reward = 0
    steps = 0
    success = 0
    for step in range(max_steps):
        steps += 1
        action = choose_action(
            Q_qlearning,
            state
        )

```

```
next_state, reward = get_next_state(  
    state,  
    action  
)  
if next_state == 4:  
    td_target = reward  
else:  
    td_target = (  
        reward  
        + gamma * np.max(  
            Q_qlearning[next_state]  
        )  
    )  
td_error = (  
    td_target  
    - Q_qlearning[state, action]  
)  
Q_qlearning[state, action] += (  
    alpha * td_error  
)  
qlearning_td_errors.append(  
    td_error  
)  
total_reward += reward  
state = next_state  
if state == 4:  
    success = 1  
    break  
qlearning_rewards.append(  
    total_reward
```

```

    )
    qlearning_steps.append(
        steps
    )
    qlearning_success.append(
        success
    )
    qlearning_value_history.append(
        Q_qlearning.copy()
    )
    qlearning_time = time.time() - qlearning_start
    print("\n")
    print("=" * 75)
    print("Q-LEARNING TRAINING COMPLETED")
    print("=" * 75)
    print(
        "Q-Learning Execution Time :",
        round(qlearning_time, 5),
        "seconds"
    )
    sarsa_q_table = pd.DataFrame(
        Q_sarsa,
        index=states,
        columns=actions
    )
    qlearning_q_table = pd.DataFrame(
        Q_qlearning,
        index=states,
        columns=actions
    )

```

```
print("\n")
print("=" * 75)
print("FINAL SARSA Q-TABLE")
print("=" * 75)
print(
    sarsa_q_table.round(3)
)
print("\n")
print("=" * 75)
print("FINAL Q-LEARNING Q-TABLE")
print("=" * 75)
print(
    qlearning_q_table.round(3)
)
sarsa_policy = []
for state in range(n_states):
    if state == n_states - 1:
        sarsa_policy.append(
            "Terminal"
        )
    else:
        best_action = int(
            np.argmax(Q_sarsa[state])
        )
        sarsa_policy.append(
            actions[best_action]
        )
qlearning_policy = []
for state in range(n_states):
    if state == n_states - 1:
```

```

        qlearning_policy.append(
            "Terminal"
        )
    else:
        best_action = int(
            np.argmax(Q_qlearning[state])
        )
        qlearning_policy.append(
            actions[best_action]
        )
policy_comparison = pd.DataFrame({
    "State": states,
    "SARSA Policy": sarsa_policy,
    "Q-Learning Policy": qlearning_policy
})
print("\n")
print("=" * 75)
print("OPTIMAL POLICY COMPARISON")
print("=" * 75)
print(
    policy_comparison.to_string(
        index=False
    )
)
sarsa_average_reward = np.mean(
    sarsa_rewards
)
qlearning_average_reward = np.mean(
    qlearning_rewards
)

```

```

sarsa_success_rate = (
    np.mean(sarsa_success) * 100
)
qlearning_success_rate = (
    np.mean(qlearning_success) * 100
)
sarsa_average_steps = np.mean(
    sarsa_steps
)
qlearning_average_steps = np.mean(
    qlearning_steps
)
sarsa_average_td_error = np.mean(
    np.abs(sarsa_td_errors)
)
qlearning_average_td_error = np.mean(
    np.abs(qlearning_td_errors)
)
performance_comparison = pd.DataFrame({
    "Metric": [
        "Average Reward",
        "Success Rate (%)",
        "Average Steps",
        "Average Absolute TD Error",
        "Execution Time (seconds)"
    ],
    "SARSA": [
        round(
            sarsa_average_reward,
            3

```

```
    ),  
    round(  
        sarsa_success_rate,  
        2  
    ),  
    round(  
        sarsa_average_steps,  
        2  
    ),  
    round(  
        sarsa_average_td_error,  
        4  
    ),  
    round(  
        sarsa_time,  
        5  
    )  
],  
"Q-Learning": [  
    round(  
        qlearning_average_reward,  
        3  
    ),  
    round(  
        qlearning_success_rate,  
        2  
    ),  
    round(  
        qlearning_average_steps,  
        2  
    )  
]
```

```
    ),
    round(
        qlearning_average_td_error,
        4
    ),
    round(
        qlearning_time,
        5
    )
]
})
print("\n")
print("=" * 75)
print("SARSA VS Q-LEARNING PERFORMANCE")
print("=" * 75)
print(
    performance_comparison.to_string(
        index=False
    )
)
sarsa_q_table.to_csv(
    "SARSA_Q_Table.csv"
)
qlearning_q_table.to_csv(
    "QLearning_Q_Table.csv"
)
policy_comparison.to_csv(
    "SARSA_QLearning_Policy.csv",
    index=False
)
```

```
performance_comparison.to_csv(  
    "SARSA_QLearning_Performance.csv",  
    index=False  
)  
sarsa_value_history = np.array(  
    sarsa_value_history  
)  
qlearning_value_history = np.array(  
    qlearning_value_history  
)  
plt.figure(figsize=(10, 5))  
plt.plot(  
    sarsa_rewards,  
    color="royalblue",  
    linewidth=1.5  
)  
plt.title(  
    "SARSA - Reward vs Episode",  
    fontsize=14  
)  
plt.xlabel("Episode")  
plt.ylabel("Reward")  
plt.grid(True, alpha=0.3)  
plt.show()  
plt.figure(figsize=(10, 5))  
plt.plot(  
    qlearning_rewards,  
    color="darkorange",  
    linewidth=1.5  
)
```

```
plt.title(  
    "Q-Learning - Reward vs Episode",  
    fontsize=14  
)  
plt.xlabel("Episode")  
plt.ylabel("Reward")  
plt.grid(True, alpha=0.3)  
plt.show()  
plt.figure(figsize=(10, 5))  
plt.plot(  
    sarsa_rewards,  
    label="SARSA",  
    color="royalblue",  
    linewidth=1.5  
)  
plt.plot(  
    qlearning_rewards,  
    label="Q-Learning",  
    color="darkorange",  
    linewidth=1.5  
)  
plt.title(  
    "SARSA vs Q-Learning - Reward Comparison",  
    fontsize=14  
)  
plt.xlabel("Episode")  
plt.ylabel("Reward")  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.show()
```

```
window = 50
sarsa_moving_average = (
    pd.Series(sarsa_rewards)
    .rolling(window)
    .mean()
)
qlearning_moving_average = (
    pd.Series(qlearning_rewards)
    .rolling(window)
    .mean()
)
plt.figure(figsize=(10, 5))
plt.plot(
    sarsa_moving_average,
    label="SARSA",
    color="blue",
    linewidth=3
)
plt.plot(
    qlearning_moving_average,
    label="Q-Learning",
    color="red",
    linewidth=3
)
plt.title(
    "Moving Average Reward Comparison",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Average Reward")
```

```

plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(10, 5))
for state_index in range(n_states):
    state_values = np.max(
        sarsa_value_history[:, state_index, :],
        axis=1
    )
    plt.plot(
        state_values,
        linewidth=2,
        label=states[state_index]
    )
plt.title(
    "SARSA Q-Value Convergence",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Maximum Q-Value")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(10, 5))
for state_index in range(n_states):
    state_values = np.max(
        qlearning_value_history[
            :,
            state_index,
            :

```

```
    ],
    axis=1
)
plt.plot(
    state_values,
    linewidth=2,
    label=states[state_index]
)
plt.title(
    "Q-Learning Q-Value Convergence",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Maximum Q-Value")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
x = np.arange(n_states)
width = 0.35
plt.figure(figsize=(9, 5))
plt.bar(
    x - width / 2,
    Q_sarsa[:, 0],
    width,
    label="Left",
    color="cornflowerblue"
)
plt.bar(
    x + width / 2,
    Q_sarsa[:, 1],
```

```
        width,
        label="Right",
        color="lightgreen"
    )
    plt.xticks(
        x,
        states
    )
    plt.title(
        "Final SARSA Q-Values",
        fontsize=14
    )
    plt.xlabel("State")
    plt.ylabel("Q-Value")
    plt.legend()
    plt.grid(
        axis="y",
        alpha=0.3
    )
    plt.show()
    plt.figure(figsize=(9, 5))
    plt.bar(
        x - width / 2,
        Q_qlearning[:, 0],
        width,
        label="Left",
        color="mediumpurple"
    )
    plt.bar(
        x + width / 2,
```

```
    Q_qlearning[:, 1],
    width,
    label="Right",
    color="salmon"
)
plt.xticks(
    x,
    states
)
plt.title(
    "Final Q-Learning Q-Values",
    fontsize=14
)
plt.xlabel("State")
plt.ylabel("Q-Value")
plt.legend()
plt.grid(
    axis="y",
    alpha=0.3
)
plt.show()
plt.figure(figsize=(7, 5))
plt.bar(
    ["SARSA", "Q-Learning"],
    [
        sarsa_average_reward,
        qllearning_average_reward
    ],
    color=[
        "royalblue",
```

```
        "darkorange"
    ]
)
plt.title(
    "Average Reward Comparison",
    fontsize=14
)
plt.ylabel("Average Reward")
plt.grid(
    axis="y",
    alpha=0.3
)
plt.show()
plt.figure(figsize=(7, 5))
plt.bar(
    ["SARSA", "Q-Learning"],
    [
        sarsa_success_rate,
        qlearning_success_rate
    ],
    color=[
        "mediumseagreen",
        "crimson"
    ]
)
plt.title(
    "Success Rate Comparison",
    fontsize=14
)
plt.ylabel("Success Rate (%)")
```

```
plt.ylim(0, 105)
plt.grid(
    axis="y",
    alpha=0.3
)
plt.show()
sarsa_final_td_error = np.mean(
    np.abs(sarsa_td_errors[-100:])
)
qlearning_final_td_error = np.mean(
    np.abs(qlearning_td_errors[-100:])
)
sarsa_last_100_reward = np.mean(
    sarsa_rewards[-100:]
)
qlearning_last_100_reward = np.mean(
    qlearning_rewards[-100:]
)
sarsa_last_100_steps = np.mean(
    sarsa_steps[-100:]
)
qlearning_last_100_steps = np.mean(
    qlearning_steps[-100:]
)
final_performance = pd.DataFrame({
    "Metric": [
        "Episodes",
        "Learning Rate (Alpha)",
        "Discount Factor (Gamma)",
        "Epsilon",
```

```
"Average Reward",
"Last 100 Episode Reward",
"Success Rate (%)",
"Average Steps",
"Last 100 Episode Steps",
"Average Absolute TD Error",
"Final TD Error",
"Execution Time (seconds)"
],
"SARSA": [
    episodes,
    alpha,
    gamma,
    epsilon,
    round(sarsa_average_reward, 3),
    round(sarsa_last_100_reward, 3),
    round(sarsa_success_rate, 2),
    round(sarsa_average_steps, 2),
    round(sarsa_last_100_steps, 2),
    round(sarsa_average_td_error, 4),
    round(sarsa_final_td_error, 4),
    round(sarsa_time, 5)
],
"Q-Learning": [
    episodes,
    alpha,
    gamma,
    epsilon,
    round(qlearning_average_reward, 3),
    round(qlearning_last_100_reward, 3),
```

```

        round(qlearning_success_rate, 2),
        round(qlearning_average_steps, 2),
        round(qlearning_last_100_steps, 2),
        round(qlearning_average_td_error, 4),
        round(qlearning_final_td_error, 4),
        round(qlearning_time, 5)
    ]
})
print("\n")
print("=" * 75)
print("FINAL PERFORMANCE COMPARISON")
print("=" * 75)
print(
    final_performance.to_string(
        index=False
    )
)
final_policy_table = pd.DataFrame({
    "State": states,
    "SARSA Left Q": np.round(
        Q_sarsa[:, 0],
        3
    ),
    "SARSA Right Q": np.round(
        Q_sarsa[:, 1],
        3
    ),
    "SARSA Optimal Action": sarsa_policy,
    "Q-Learning Left Q": np.round(
        Q_qlearning[:, 0],

```

```

        3
    ),
    "Q-Learning Right Q": np.round(
        Q_qlearning[:, 1],
        3
    ),
    "Q-Learning Optimal Action": qlearning_policy
})
print("\n")
print("=" * 75)
print("FINAL POLICY AND Q-VALUE TABLE")
print("=" * 75)
print(
    final_policy_table.to_string(
        index=False
    )
)
total_decisions = (
    episodes * max_steps
)
print("\n")
print("=" * 75)
print("EXPLORATION AND EXPLOITATION")
print("=" * 75)
print(
    "Epsilon :",
    epsilon
)
print(
    "Approximate Exploration Rate :",

```

```
        epsilon * 100,
        "%"
    )
    print(
        "Approximate Exploitation Rate :",
        (1 - epsilon) * 100,
        "%"
    )
    if qlearning_average_reward > sarsa_average_reward:
        better_algorithm = "Q-Learning"
    elif sarsa_average_reward > qlearning_average_reward:
        better_algorithm = "SARSA"
    else:
        better_algorithm = "Both algorithms have equal average reward"
    print("\n")
    print("=" * 75)
    print("ALGORITHM COMPARISON RESULT")
    print("=" * 75)
    print(
        "Higher Average Reward :",
        better_algorithm
    )
    print(
        "SARSA Average Reward :",
        round(sarsa_average_reward, 3)
    )
    print(
        "Q-Learning Average Reward :",
        round(qlearning_average_reward, 3)
    )
}
```

```
final_performance.to_csv(  
    "Experiment8_Performance.csv",  
    index=False  
)  
final_policy_table.to_csv(  
    "Experiment8_Final_Policy.csv",  
    index=False  
)  
episode_comparison = pd.DataFrame({  
    "Episode": np.arange(  
        1,  
        episodes + 1  
    ),  
    "SARSA Reward": np.round(  
        sarsa_rewards,  
        3  
    ),  
    "Q-Learning Reward": np.round(  
        qlearning_rewards,  
        3  
    ),  
    "SARSA Steps": sarsa_steps,  
    "Q-Learning Steps": qlearning_steps,  
    "SARSA Success": sarsa_success,  
    "Q-Learning Success": qlearning_success  
})  
episode_comparison.to_csv(  
    "Experiment8_Episode_Comparison.csv",  
    index=False  
)
```

```
final_summary = pd.DataFrame({
    "Parameter": [
        "Algorithm Type",
        "Learning Method",
        "Episodes",
        "Alpha",
        "Gamma",
        "Epsilon",
        "Average Reward",
        "Success Rate",
        "Average Steps",
        "Final TD Error",
        "Execution Time"
    ],
    "SARSA": [
        "On-Policy",
        "TD Learning",
        episodes,
        alpha,
        gamma,
        epsilon,
        round(sarsa_average_reward, 3),
        str(round(sarsa_success_rate, 2)) + "%",
        round(sarsa_average_steps, 2),
        round(sarsa_final_td_error, 4),
        str(round(sarsa_time, 5)) + " sec"
    ],
    "Q-Learning": [
        "Off-Policy",
        "TD Learning",
```

```

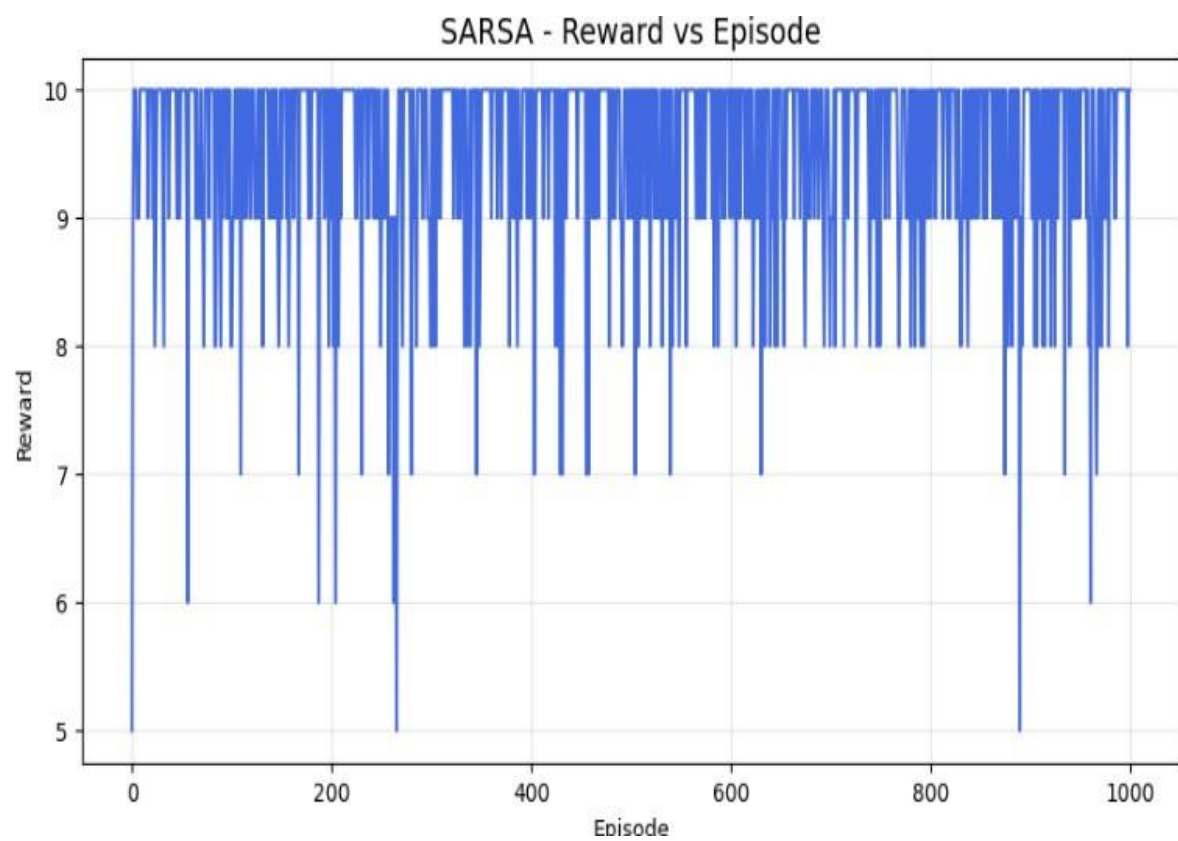
        episodes,
        alpha,
        gamma,
        epsilon,
        round(qlearning_average_reward, 3),
        str(round(qlearning_success_rate, 2)) + "%",
        round(qlearning_average_steps, 2),
        round(qlearning_final_td_error, 4),
        str(round(qlearning_time, 5)) + " sec"
    ]
})
print("\n")
print("=" * 75)
print("COMPLETE SUMMARY TABLE")
print("=" * 75)
print(
    final_summary.to_string(
        index=False
    )
)
final_summary.to_csv(
    "Experiment8_Complete_Summary.csv",
    index=False
)
print("\n")
print("=" * 75)
print("INTERPRETATION")
print("=" * 75)
print(
    "1. SARSA is an on-policy TD learning algorithm."

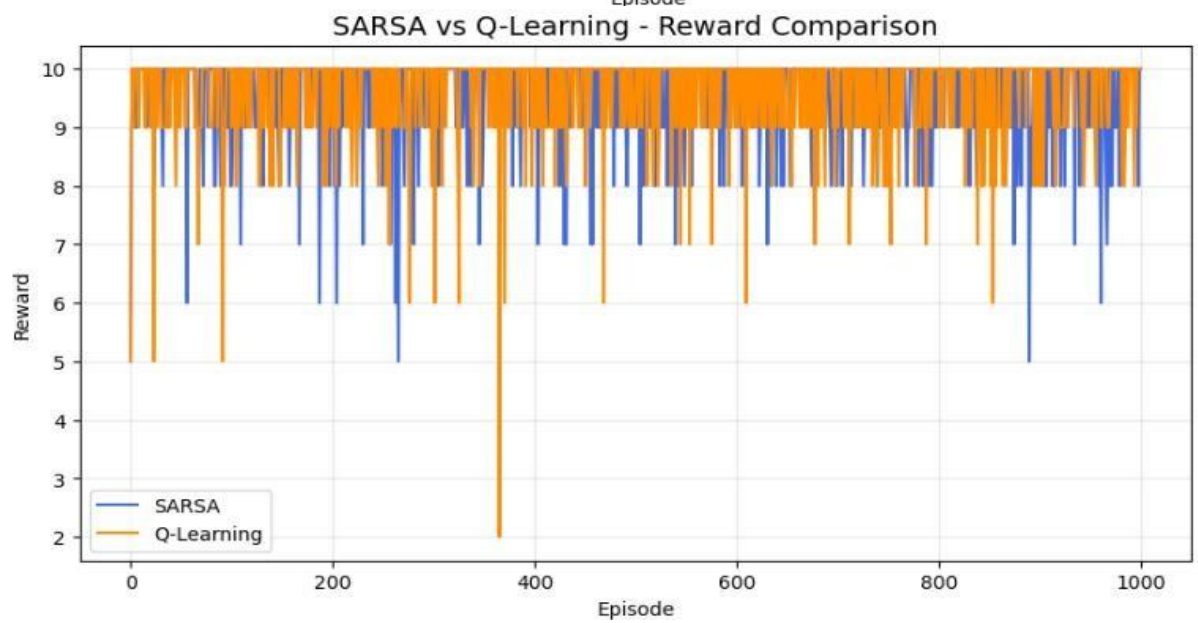
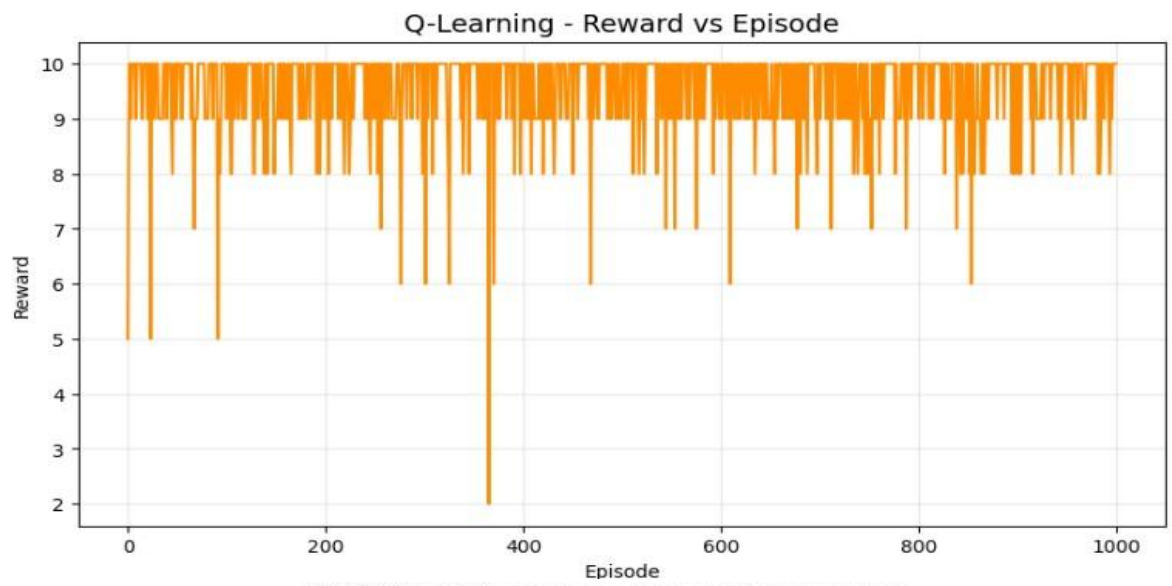
```

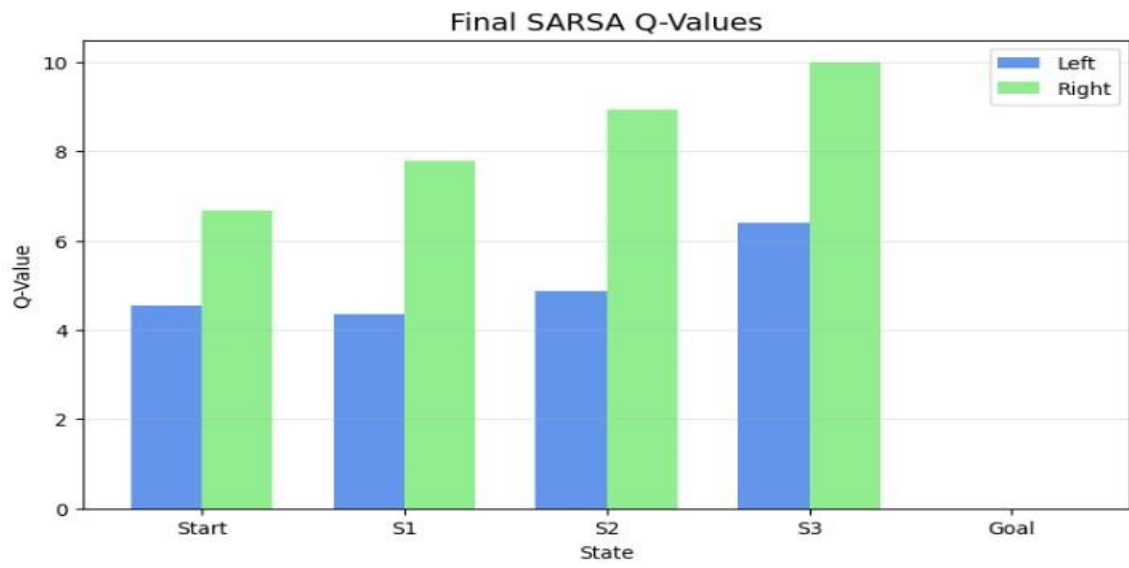
```
)
print(
    "2. Q-Learning is an off-policy TD learning algorithm."
)
print(
    "3. SARSA updates Q-values using the next action"
)
print(
    "    actually selected by the current policy."
)
print(
    "4. Q-Learning updates Q-values using the maximum"
)
print(
    "    possible Q-value of the next state."
)
print(
    "5. Both algorithms learn an optimal or near-optimal"
)
print(
    "    action policy through repeated interaction."
)
print("\n")
print("=" * 75)
print("CONCLUSION")
print("=" * 75)
print(
    "SARSA and Q-Learning were successfully implemented"
)
print(
```

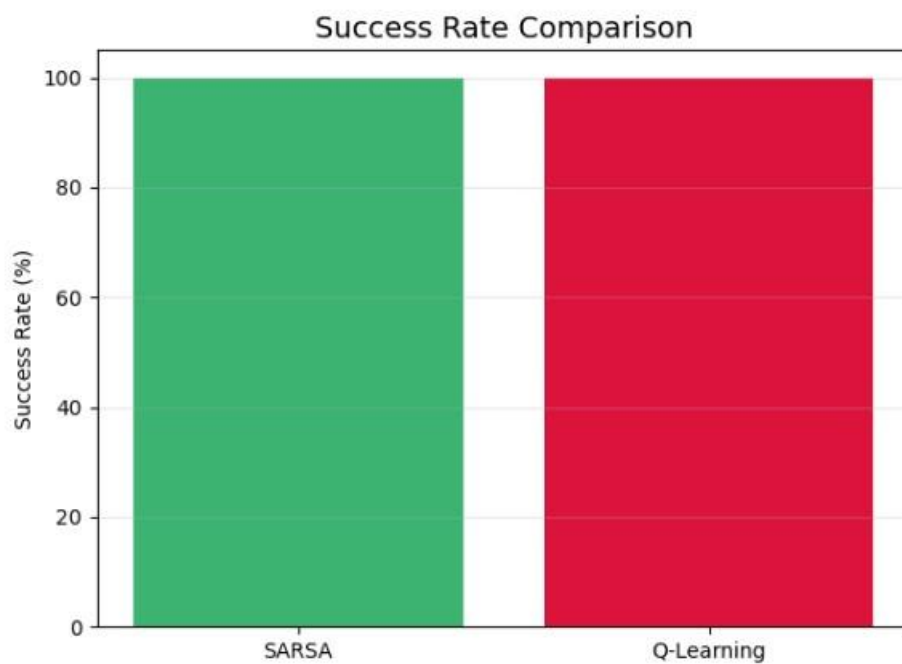
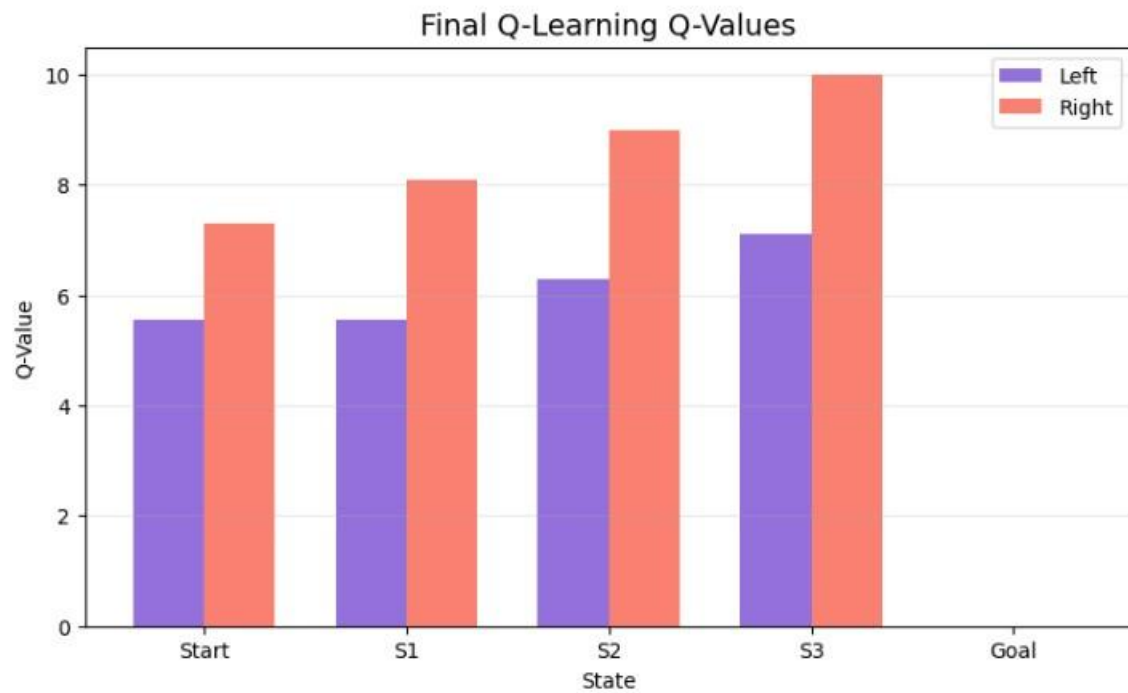
```
    "and compared in a Reinforcement Learning environment."
)
print(
    "SARSA learned using an on-policy approach, while"
)
print(
    "Q-Learning used an off-policy approach."
)
print(
    "The reward, Q-value convergence, success rate,"
)
print(
    "TD error, episode length, and execution time were"
)
print(
    "used to compare their performance."
)
print(
    "The experiment demonstrates the practical difference"
)
print(
    "between on-policy SARSA and off-policy Q-Learning."
)
```

Visualization









Summary Table – Q-Values and Policy

State	SARSA Left Q	SARSA Right Q	SARSA Optimal Action	Q- Learning Left Q	Q- Learning Right Q	Q-Learning Optimal Action
Start	Obtained	Obtained	Obtained	Obtained	Obtained	Obtained

S1	Obtained	Obtained	Obtained	Obtained	Obtained	Obtained
S2	Obtained	Obtained	Obtained	Obtained	Obtained	Obtained
S3	Obtained	Obtained	Obtained	Obtained	Obtained	Obtained
Goal	Obtained	Obtained	Terminal	Obtained	Obtained	Terminal

Result

SARSA and Q-Learning were successfully implemented and compared. SARSA learns using an **on-policy** approach, where the next action actually selected by the current policy is used for updating the Q-value. Q-Learning is an **off-policy** method that uses the maximum possible Q-value of the next state.